

How JS Work Behind the Scene



Eng. Nadia Saleh

JAVASCRIPT

JAVASCRIPT IS A HIGH-LEVEL, PROTOTYPE-BASED OBJECT-ORIENTED, MULTI-PARADIGM, INTERPRETED OR JUST-IN-TIME COMPILED, DYNAMIC, SINGLE-THREADED, GARBAGE-COLLECTED PROGRAMMING LANGUAGE WITH FIRST-CLASS FUNCTIONS AND A NON-BLOCKING EVENT LOOP CONCURRENCY MODEL. 🤔 🍷 🤪

JS

High-level

Garbage-collected

Interpreted or just-in-time compiled

Multi-paradigm

Prototype-based object-oriented

First-class functions

Dynamic

Single-threaded

Non-blocking event loop

👉 **Paradigm:** An approach and mindset of structuring code, which will direct your coding style and technique.

 The one we've been
using so far

- 1 Procedural programming
- 2 Object-oriented programming (OOP)
- 3 Functional programming (FP)

- 👉 **Concurrency model:** how the JavaScript engine handles multiple tasks happening at the same time.



Why do we need that?

- 👉 JavaScript runs in one **single thread**, so it can only do one thing at a time.



So what about a long-running task?

- 👉 Sounds like it would block the single thread. However, we want non-blocking behavior!



How do we achieve that?

(Oversimplification!)

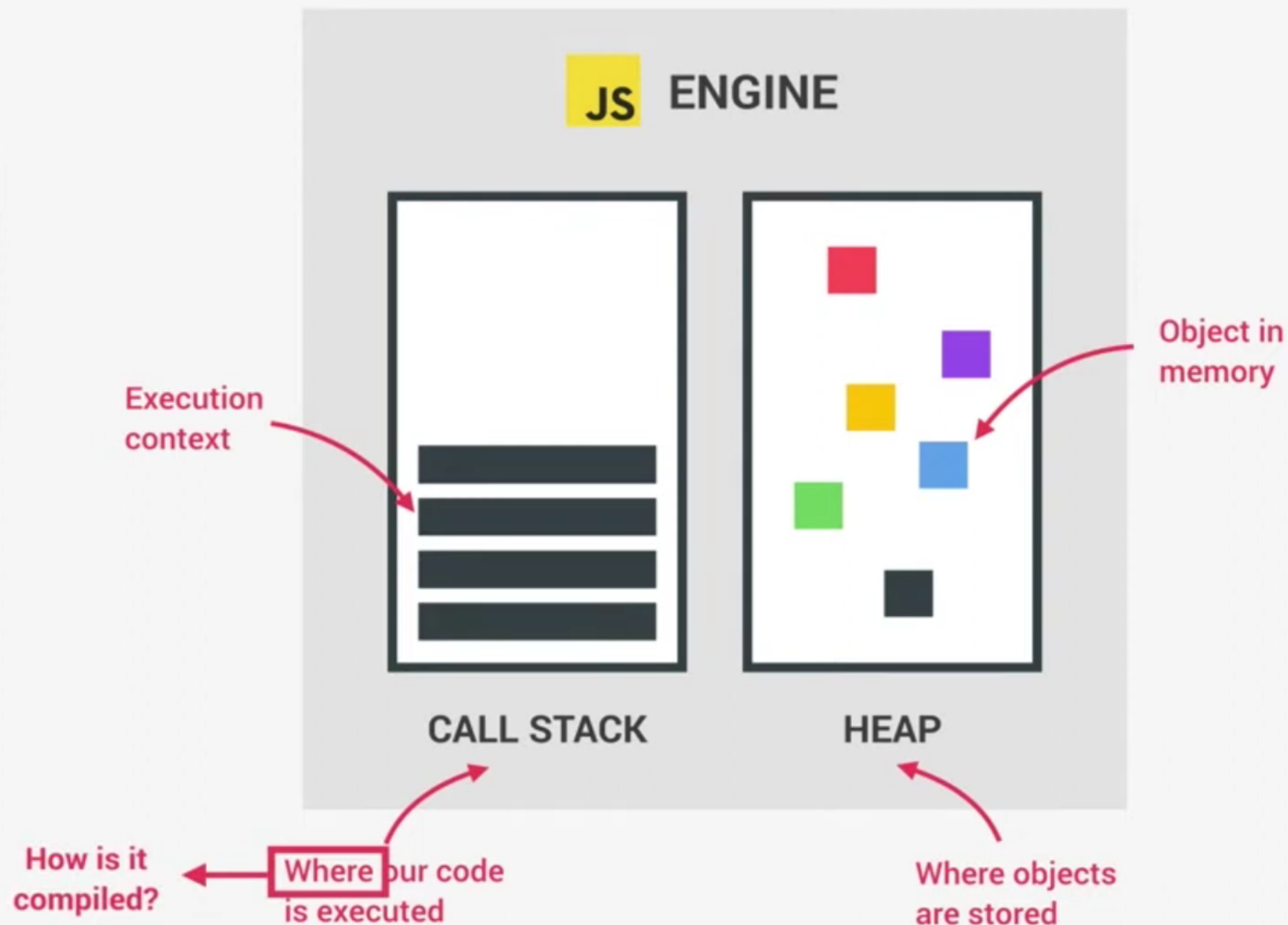
- 👉 By using an **event loop**: takes long running tasks, executes them in the "background", and puts them back in the main thread once they are finished.

WHAT IS A JAVASCRIPT ENGINE?

JS ENGINE

PROGRAM THAT **EXECUTES**
JAVASCRIPT CODE.

👉 Example: V8 Engine

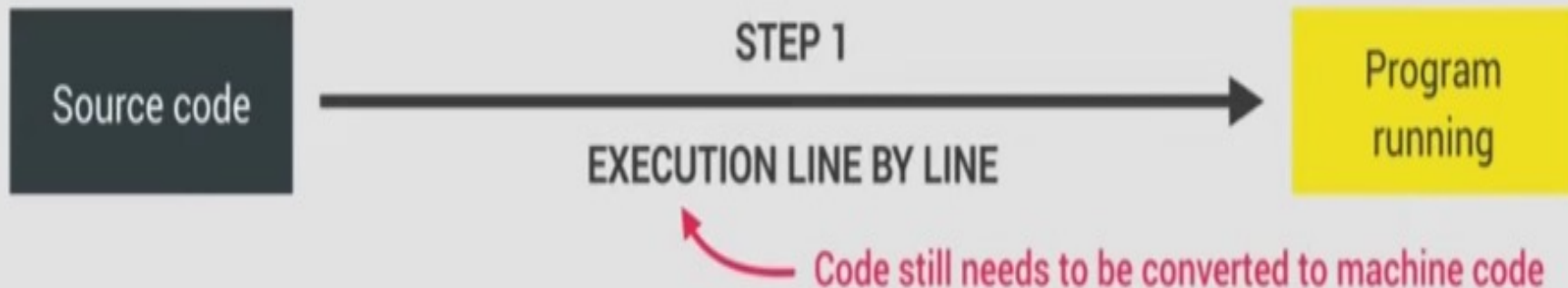


IS JavaScript Interpreted or Compiled ?

👉 **Compilation:** Entire code is converted into machine code at once, and written to a binary file that can be executed by a computer.

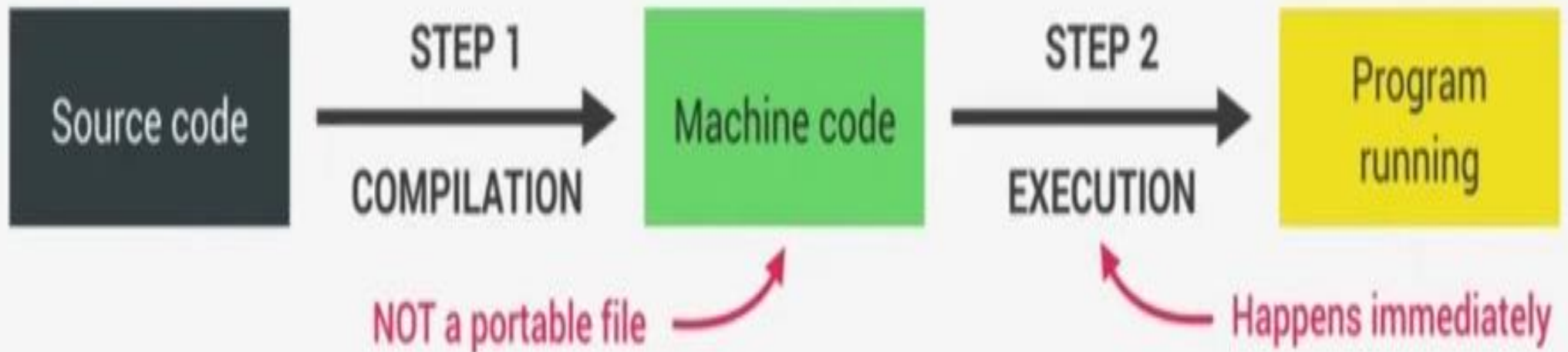


👉 **Interpretation:** Interpreter runs through the source code and executes it line by line.

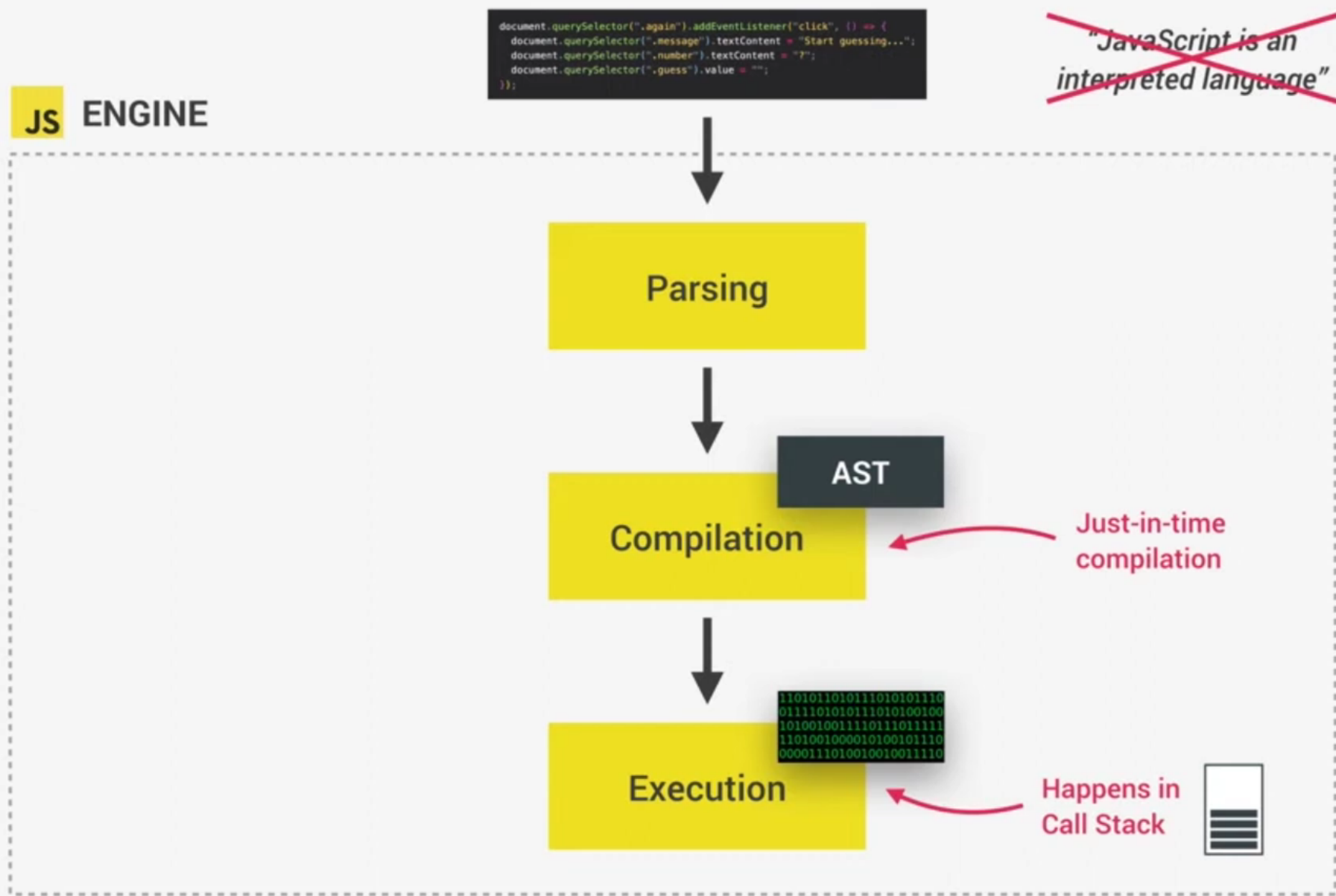


IS JavaScript Interpreted or Compiled ?

👉 **Just-in-time (JIT) compilation:** Entire code is converted into machine code at once, then executed immediately.



MODERN JUST-IN-TIME COMPILATION OF JAVASCRIPT



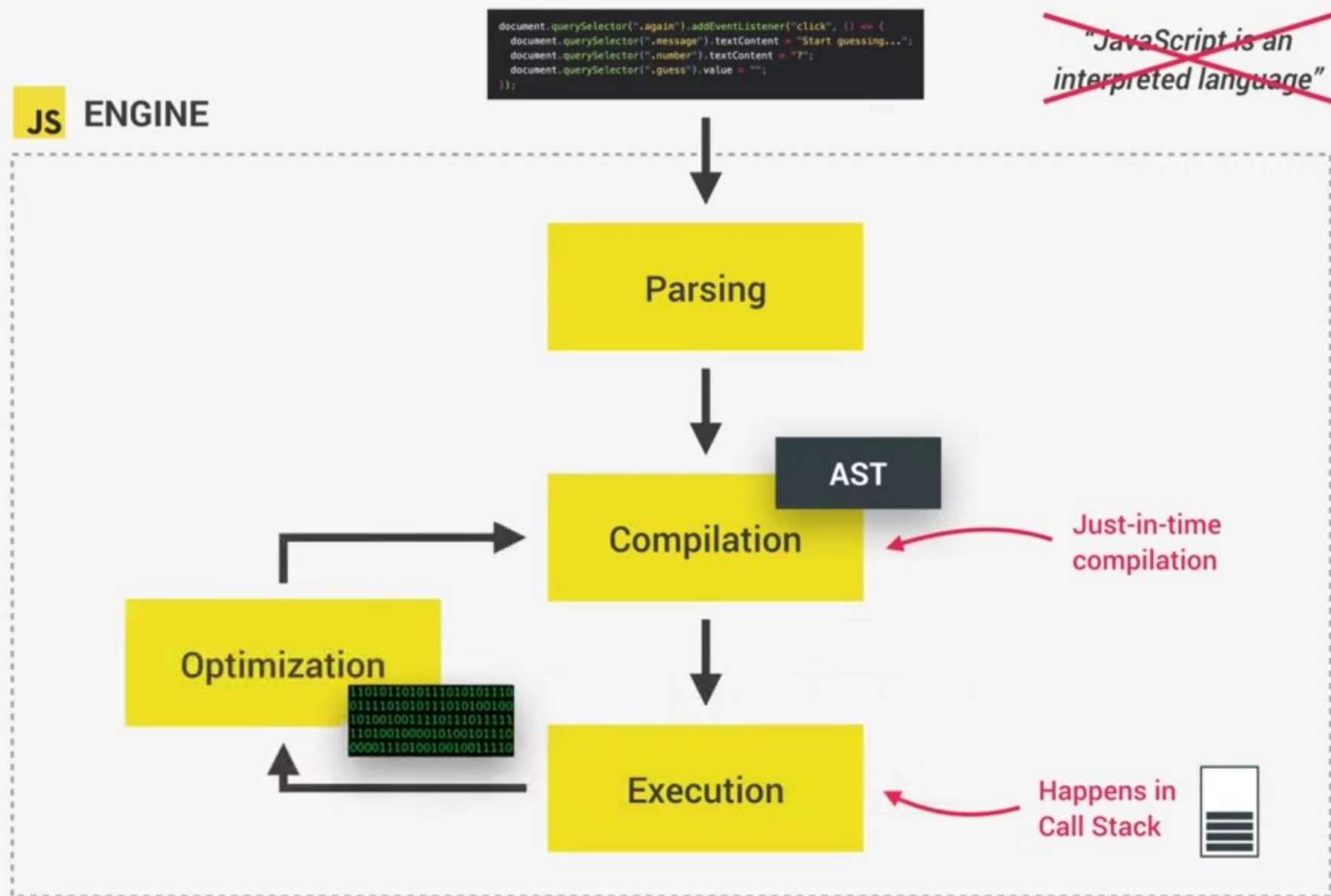
~~"JavaScript is an interpreted language"~~

👉 AST Example

```
const x = 23;
```

```
- VariableDeclaration {
  start: 0
  end: 13
  declarations: [
    - VariableDeclarator {
      start: 6
      end: 12
      id: Identifier {
        start: 6
        end: 7
        name: "x"
      }
      init: Literal = $node {
        start: 10
        end: 12
        value: 23
        raw: "23"
      }
    }
  ]
  kind: "const"
}
```

MODERN JUST-IN-TIME COMPILATION OF JAVASCRIPT

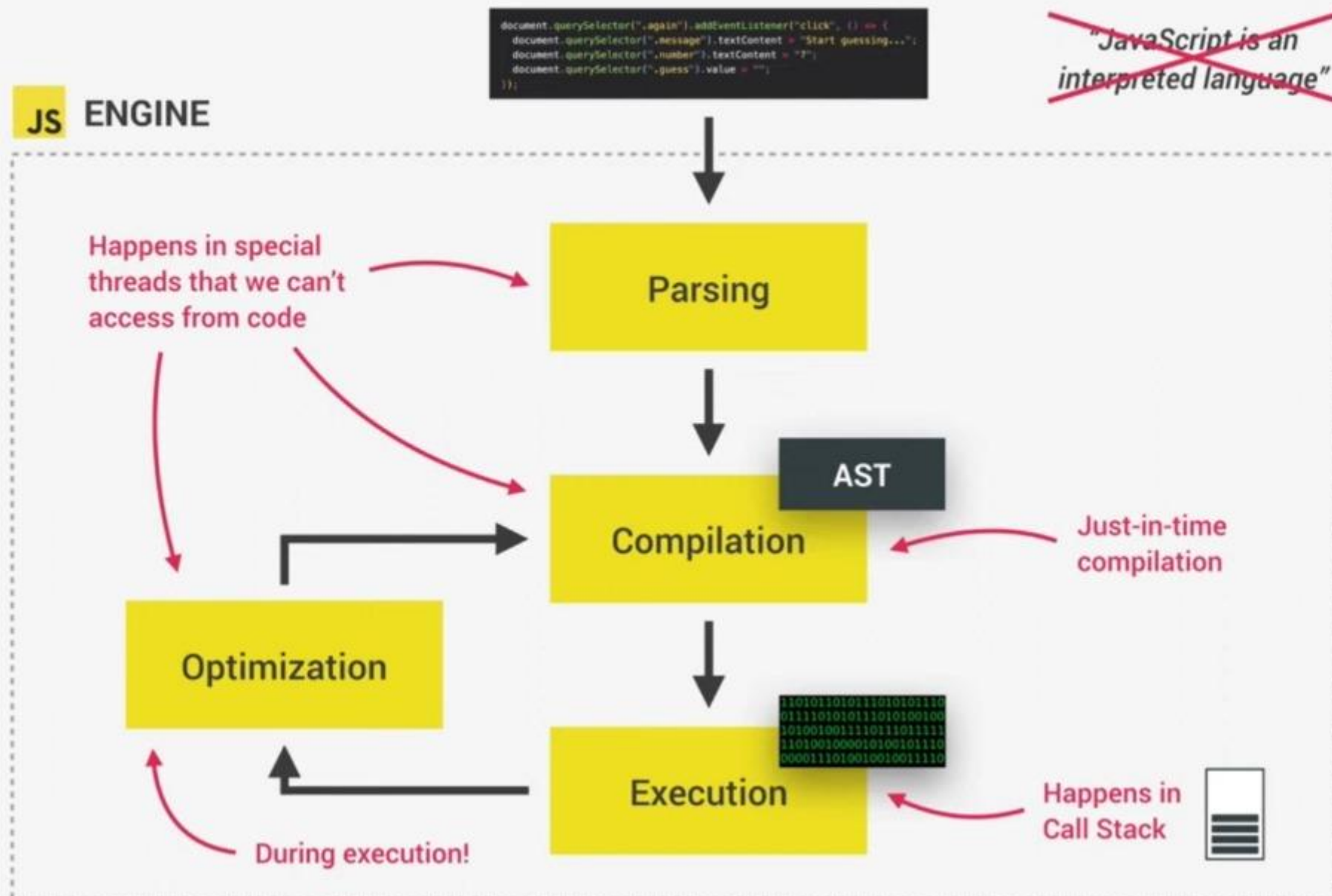


AST Example

```
const x = 23;
```

```
- VariableDeclaration {
  start: 0
  end: 13
  declarations: [
    - VariableDeclarator {
      start: 6
      end: 12
      id: Identifier {
        start: 6
        end: 7
        name: "x"
      }
      init: Literal = $node {
        start: 10
        end: 12
        value: 23
        raw: "23"
      }
    }
  ]
}
kind: "const"
```

MODERN JUST-IN-TIME COMPILATION OF JAVASCRIPT



AST Example

```
const x = 23;
```

```
- VariableDeclaration {
  start: 0
  end: 13
  - declarations: [
    - VariableDeclarator {
      start: 6
      end: 12
      - id: Identifier {
        start: 6
        end: 7
        name: "x"
      }
      - init: Literal = $node {
        start: 10
        end: 12
        value: 23
        raw: "23"
      }
    }
  ]
  kind: "const"
}
```

IS JavaScript Interpreted or Compiled ?

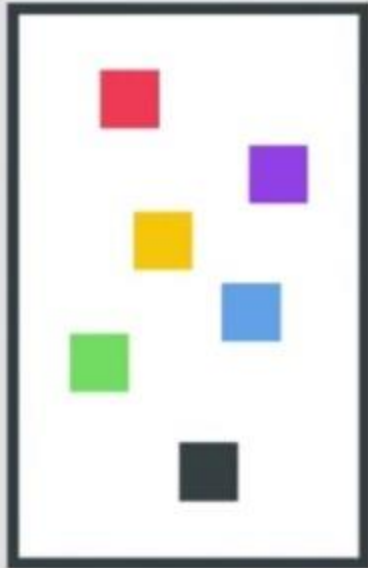
An **Abstract Syntax Tree (AST)** is a tree-like data structure that represents the hierarchical structure of our code. It is a crucial intermediate step in the process of executing JavaScript within the V8 engine. The AST is generated during parsing and is later used for interpretation and optimization.



RUNTIME IN THE BROWSER



ENGINE



HEAP



CALL STACK

WEB APIs

DOM

Timers

Fetch API

...

Functionalities provided to the engine, accessible on window object

CALLBACK QUEUE

onClick

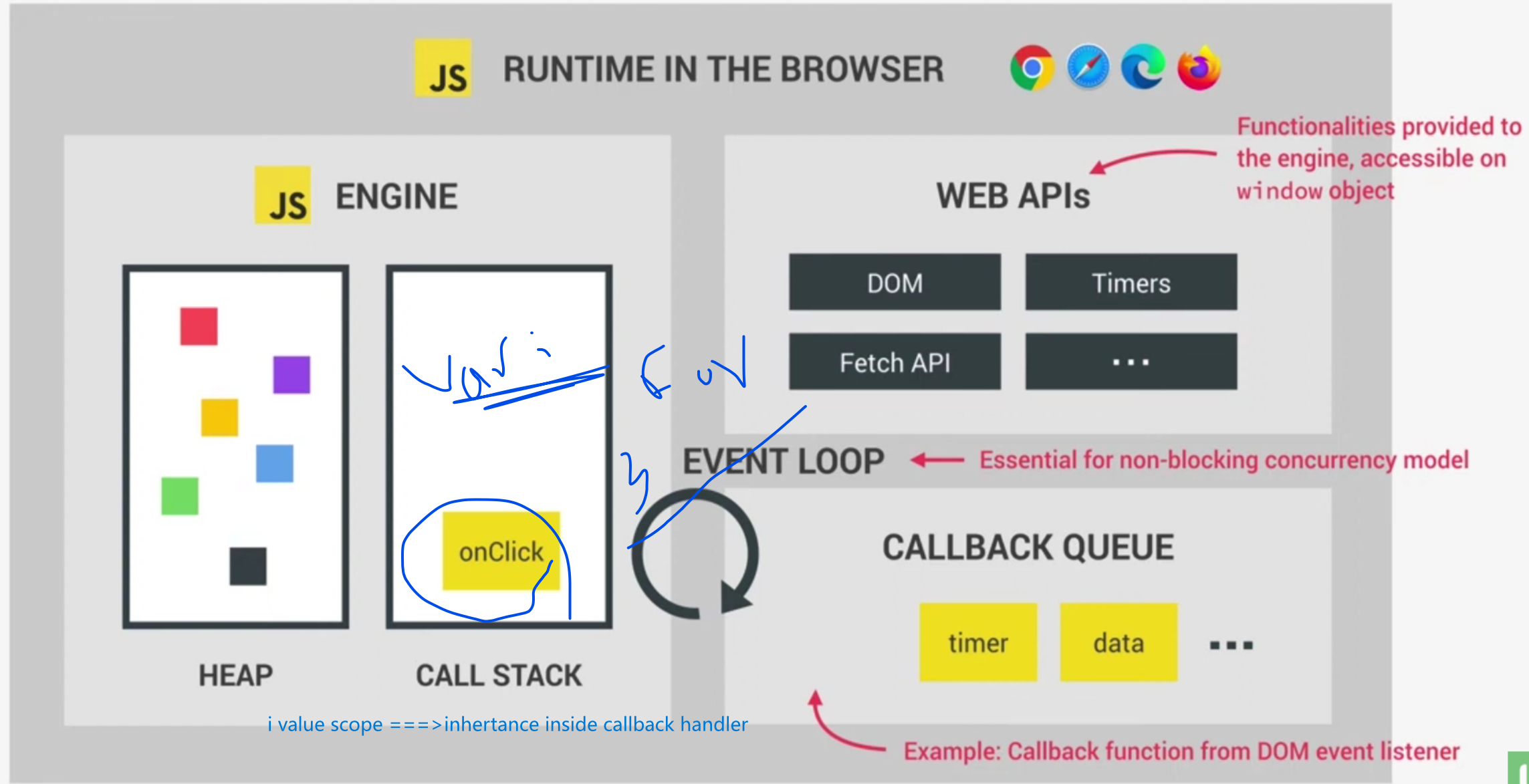
timer

data

...

Example: Callback function from DOM event listener

THE BIGGER PICTURE: JAVASCRIPT RUNTIME



WHAT IS AN EXECUTION CONTEXT?

👉 Human-readable code:

```
const name = 'Jonas';
```

```
const first = () => {  
  let a = 1;  
  const b = second();  
  a = a + b;  
  return a;  
};
```

```
function second() {  
  var c = 2;  
  return c;  
}
```

Function body
only executed
when called!

EXECUTION

Compilation

Creation of **global execution context** (for top-level code)

```
1101011010111010101110  
0111101010111010100100  
1010010011110111011111  
1101001000010100101110  
0000111010010010011110
```

NOT inside
a function

Execution of **top-level code**
(inside global EC)

Execution of **functions** and
waiting for **callbacks**



Example: click event callback

EXECUTION CONTEXT

Environment in which a piece of JavaScript is executed. Stores all the necessary information for some code to be executed.



- 👉 Exactly **one** global execution context (EC): Default context, created for code that is not inside any function (top-level).
- 👉 One execution context **per function**: For each function call, a new execution context is created.

All together make the call stack

EXECUTION CONTEXT IN DETAIL

WHAT'S INSIDE EXECUTION CONTEXT?

1 Variable Environment

- 👉 let, const and var declarations
- 👉 Functions
- 👉 ~~arguments~~ object

2 Scope chain

3 ~~this~~ keyword

NOT in arrow functions!

Generated during "creation phase", right before execution

```
const name = 'Jonas';

const first = () => {
  let a = 1;
  const b = second(7, 9);
  a = a + b;
  return a;
};

function second(x, y) {
  var c = 2;
  return c;
}

const x = first();
```

Global

```
name = 'Jonas'
first = <function>
second = <function>
x = <unknown>
```

Literally the function code

Need to run first() first

first()

```
a = 1
b = <unknown>
```

Need to run second() first

second()

```
c = 2
arguments = [7, 9]
```

Array of passed arguments. Available in all "regular" functions (not arrow)

(Technically, values only become known during execution)

THE CALL STACK

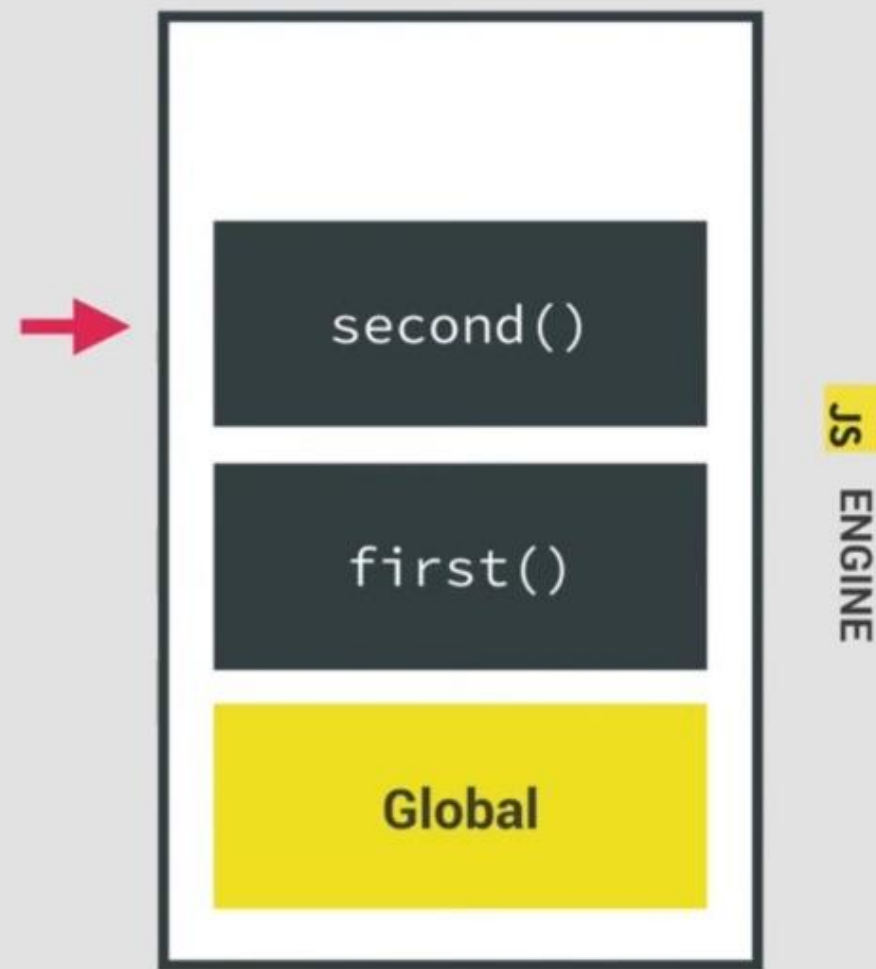
👉 Compiled code starts execution

```
const name = 'Jonas';

const first = () => {
  let a = 1;
  const b = second(7, 9);
  a = a + b;
  return a;
};

function second(x, y) {
  var c = 2;
  return c;
}

const x = first();
```



"Place" where execution contexts get stacked on top of each other, to keep track of where we are in the execution

THE 3 TYPES OF SCOPE

GLOBAL SCOPE

```
const me = 'Jonas';  
const job = 'teacher';  
const year = 1989;
```

- 👉 Outside of **any** function or block
- 👉 Variables declared in global scope are accessible **everywhere**

FUNCTION SCOPE

```
function calcAge(birthYear) {  
  const now = 2037;  
  const age = now - birthYear;  
  return age;  
}  
  
console.log(now); // ReferenceError
```

- 👉 Variables are accessible only **inside function, NOT** outside
- 👉 Also called local scope

BLOCK SCOPE (ES6)

```
if (year >= 1981 && year <= 1996) {  
  const millenial = true;  
  const food = 'Avocado toast';  
} ← Example: if block, for loop block, etc.  
  
console.log(millenial); // ReferenceError
```

- 👉 Variables are accessible only **inside block** (block scoped)
- ⚠️ **HOWEVER**, this only applies to **let** and **const** variables!
- 👉 Functions are **also block scoped** (only in strict mode)

SCOPING AND SCOPE IN JAVASCRIPT: CONCEPTS

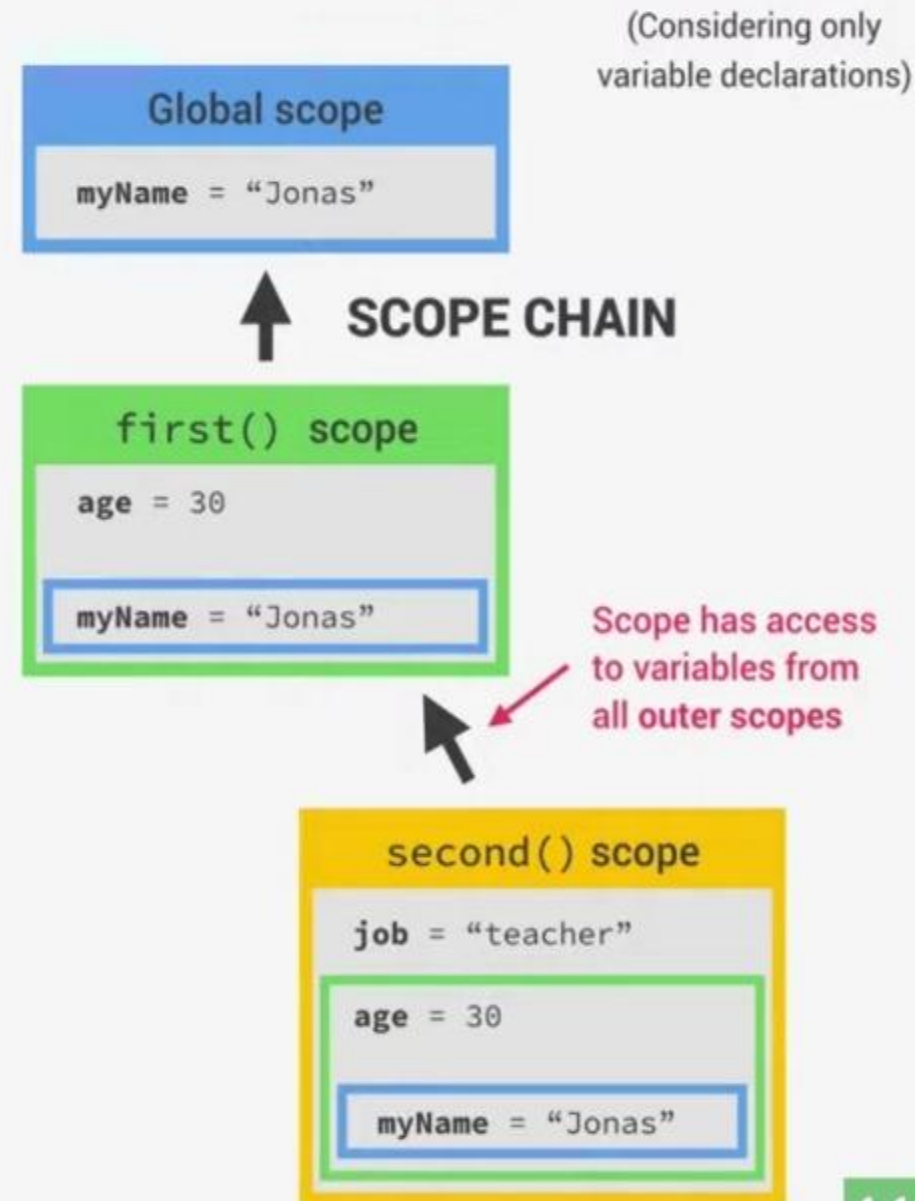
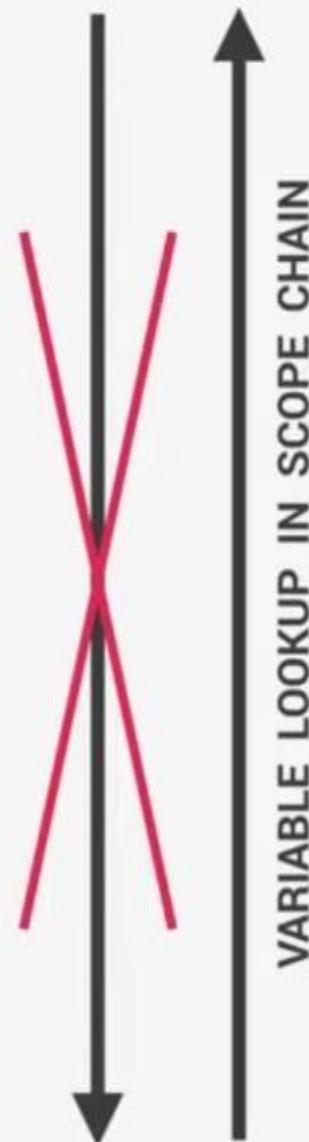
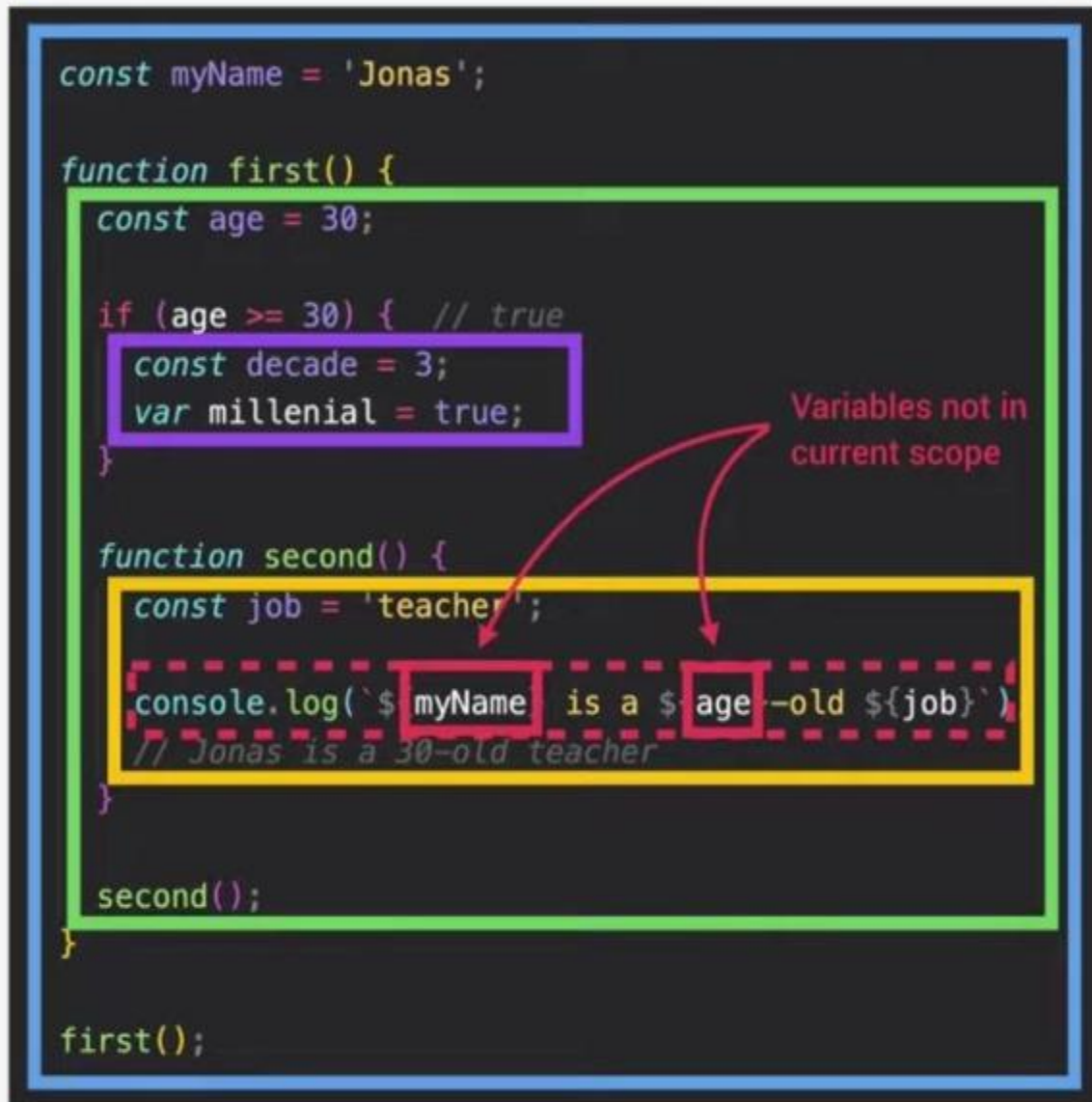
SCOPE CONCEPTS

EXECUTION CONTEXT

- 👉 Variable environment
- 👉 Scope chain
- 👉 this keyword

- 👉 **Scoping:** How our program's variables are **organized** and **accessed**. *"Where do variables live?" or "Where can we access a certain variable, and where not?"*
- 👉 **Lexical scoping:** Scoping is controlled by **placement** of functions and blocks in the code;
- 👉 **Scope:** Space or environment in which a certain variable is **declared** (*variable environment in case of functions*). There is **global** scope, **function** scope, and **block** scope;
- 👉 **Scope of a variable:** Region of our code where a certain variable can be **accessed**.

THE SCOPE CHAIN



SCOPE CHAIN VS. CALL STACK

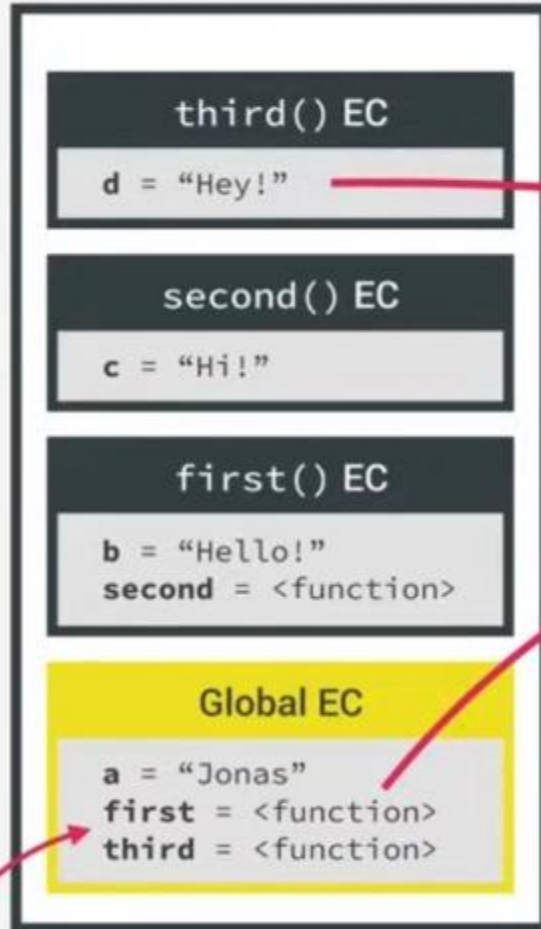
```
const a = 'Jonas';  
first();  
  
function first() {  
  const b = 'Hello!';  
  second();  
  
  function second() {  
    const c = 'Hi!';  
    third();  
  }  
}  
  
function third() {  
  const d = 'Hey!';  
  console.log(d + c + b + a);  
  // ReferenceError  
}
```

c and b can NOT be found
in third() scope!

Variable
environment (VE)

CALL STACK

Order in which
functions were **called**



third() VE

+

Global VE

Global scope

a = "Jonas"
first = <function>
third = <function>

first() scope

b = "Hello!"
second = <function>

a = "Jonas"
first = <function>
third = <function>

third() scope

d = "Hey!"

a = "Jonas"
first = <function>
third = <function>

SCOPE CHAIN

Order in which functions
are **written in the code**

👉 Has **nothing** to do
with order in which
functions were **called**!

HOISTING IN JAVASCRIPT

👉 **Hoisting:** Makes some types of variables accessible/usable in the code before they are actually declared. "Variables lifted to the top of their scope".

↓
BEHIND THE SCENES

Before execution, code is scanned for variable declarations, and for each variable, a new property is created in the **variable environment object**.

EXECUTION CONTEXT

👉 Variable environment

✅ Scope chain

👉 this keyword

	HOISTED? 👉	INITIAL VALUE 👉	SCOPE 👉
function declarations	✅ YES	Actual function	Block
var variables	✅ YES	undefined	Function
let and const variables	🚫 NO	<uninitialized>, TDZ	Block
function expressions and arrows	🤖 Depends if using var or let/const		

Technically, yes. But not in practice

In strict mode. Otherwise: function!

Temporal Dead Zone

TEMPORAL DEAD ZONE, LET AND CONST

```
const myName = 'Jonas';  
  
if (myName === 'Jonas') {  
  console.log(`Jonas is a ${job}`);  
  const age = 2037 - 1989;  
  console.log(age);  
  const job = 'teacher';  
  console.log(x);  
}
```

TEMPORAL DEAD ZONE FOR **job** VARIABLE

👉 Different kinds of error messages:

ReferenceError: Cannot access 'job' before initialization

ReferenceError: x is not defined

WHY HOISTING?

- 👉 Using functions before actual declaration;
- 👉 var hoisting is just a byproduct.

WHY TDZ?

- 👉 **Makes it easier to avoid and catch errors:** accessing variables before declaration is bad practice and should be avoided;
- 👉 **Makes const variables actually work**

HOW THE THIS KEYWORD WORKS

- 👉 **this keyword/variable:** Special variable that is created for every execution context (every function). Takes the value of (points to) the "owner" of the function in which the **this** keyword is used.
- 👉 **this** is **NOT** static. It depends on **how** the function is called, and its value is only assigned when the function is **actually called**.

EXECUTION CONTEXT

- ✓ Variable environment
- ✓ Scope chain
- 👉 **this keyword**

Method 👉 **this** = <Object that is calling the method>

Simple function call 👉 **this** = undefined

In strict mode! Otherwise:
window (in the browser)

Don't get
own this

Arrow functions 👉 **this** = <this of surrounding function (lexical this)>

Event listener 👉 **this** = <DOM element that the handler is attached to>

new, call, apply, bind 👉 <Later in the course... ⌚>

Method example:

```
const jonas = {  
  name: 'Jonas',  
  year: 1989,  
  calcAge: function() {  
    return 2037 - this.year  
  }  
};  
jonas.calcAge(); // 48
```

calcAge
is method

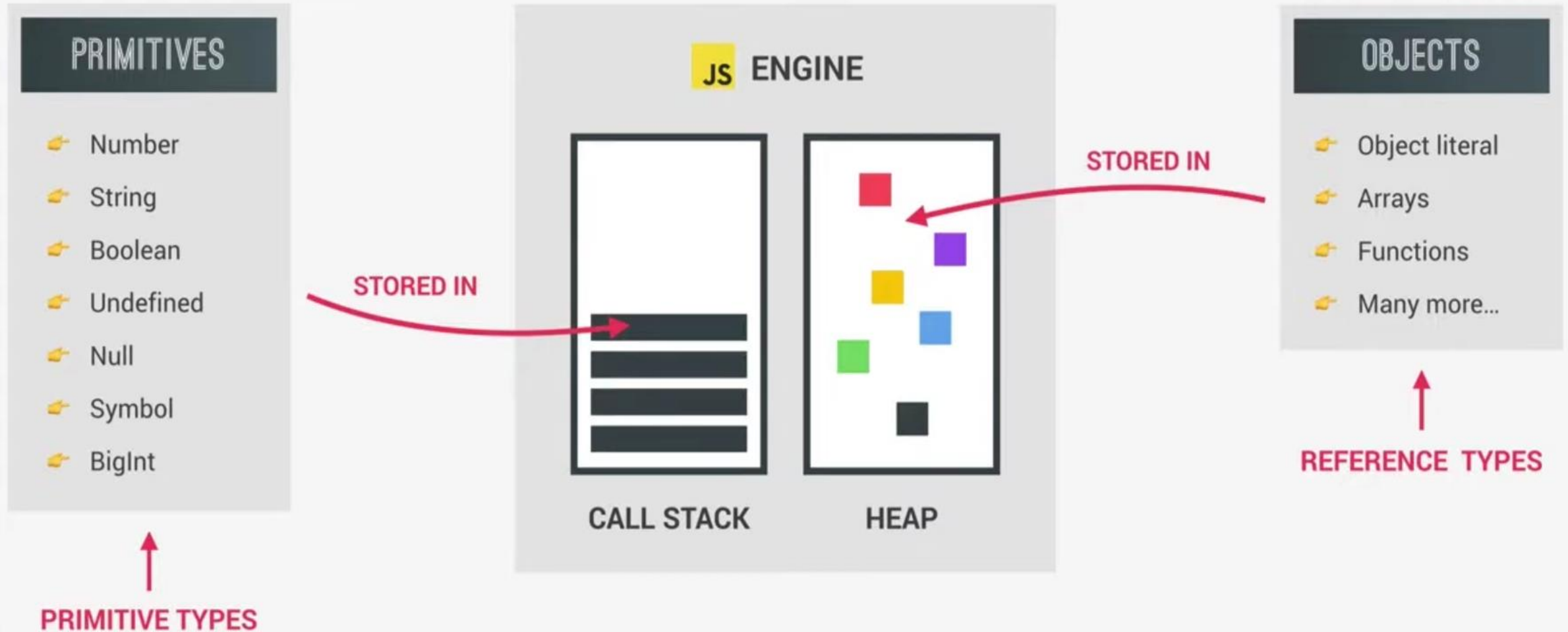
jonas

1989

Way better than using
jonas.year!

- 👉 **this** does **NOT** point to the function itself, and also **NOT** the its variable environment!

REVIEW: PRIMITIVES, OBJECTS AND THE JAVASCRIPT ENGINE



PRIMITIVE VS. REFERENCE VALUES

👉 Primitive values example:

```
let age = 30;
let oldAge = age;
age = 31;
console.log(age); // 31
console.log(oldAge); // 30
```

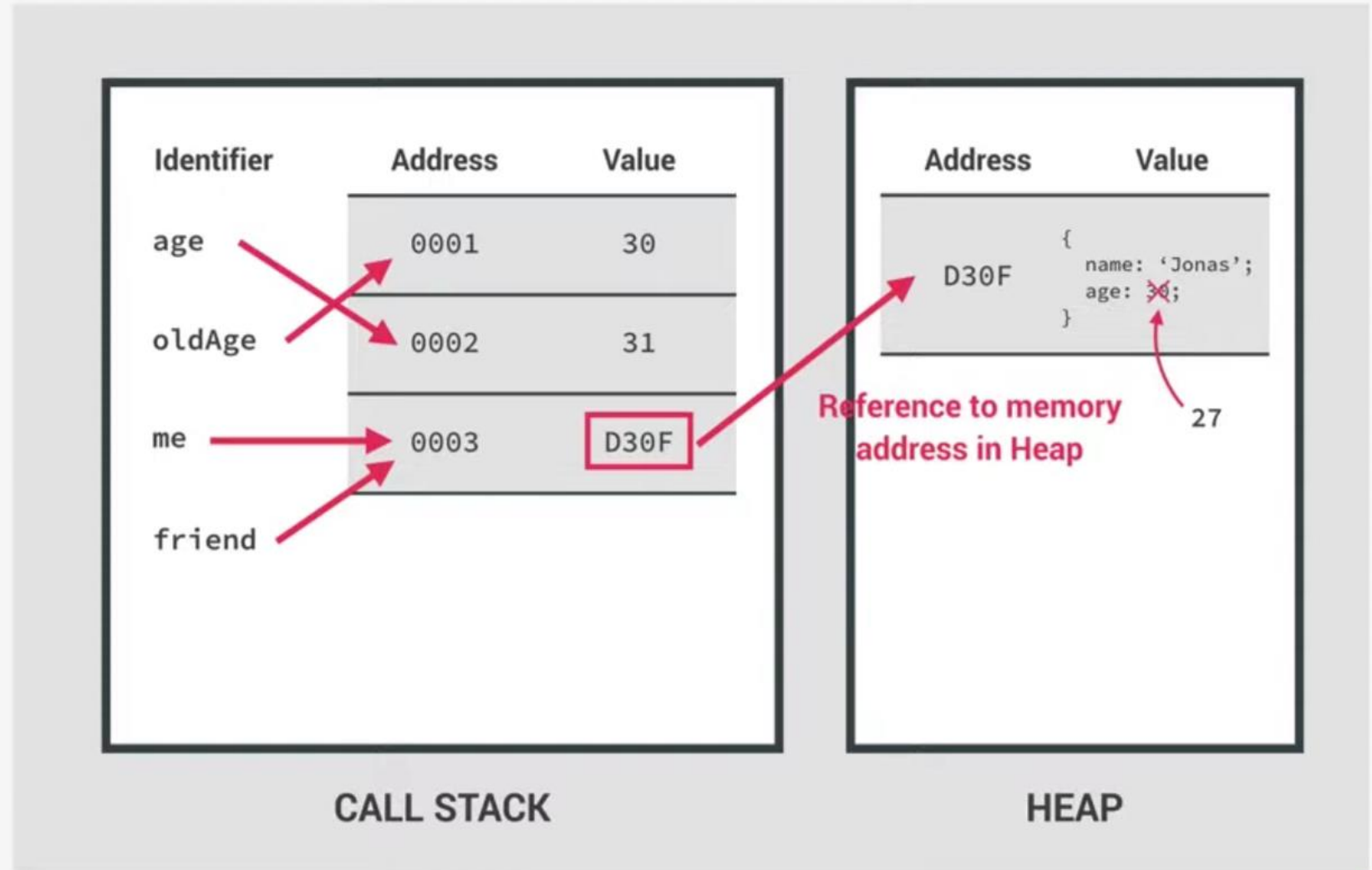
👉 Reference values example:

```
const me = {
  name: 'Jonas',
  age: 30
};
const friend = me;
friend.age = 27;

console.log('Friend:', friend);
// { name: 'Jonas', age: 27 }

console.log('Me:', me);
// { name: 'Jonas', age: 27 }
```

No problem, because we're NOT changing the value at address 0003!



JS Functional Programming

Eng. Nadia Saleh

A large, stylized 'JS' logo in white with a subtle gradient and a long shadow. The 'J' and 'S' are connected. The background is a gradient from dark yellow to light yellow.

FIRST-CLASS VS. HIGHER-ORDER FUNCTIONS

FIRST-CLASS FUNCTIONS

- 👉 JavaScript treats functions as **first-class citizens**
- 👉 This means that functions are **simply values**
- 👉 Functions are just another **"type" of object**

- 👉 Store functions in variables or properties:

```
const add = (a, b) => a + b;  
  
const counter = {  
  value: 23,  
  inc: function() { this.value++; }  
};
```

- 👉 Pass functions as arguments to OTHER functions:

```
const greet = () => console.log('Hev Jonas');  
btnClose.addEventListener('click', greet);
```

- 👉 Return functions FROM functions

- 👉 Call methods on functions:

```
counter.inc.bind(someOtherObject);
```

HIGHER-ORDER FUNCTIONS

- 👉 A function that **receives** another function as an argument, that **returns** a new function, or **both**
- 👉 This is only possible because of first-class functions

1 Function that receives another function

```
const greet = () => console.log('Hev Jonas');  
btnClose.addEventListener('click', greet);
```

Higher-order
function

Callback
function



2 Function that returns new function

```
function count() {  
  let counter = 0;  
  return function() {  
    counter++;  
  };  
}
```

Higher-order
function

Returned
function

"CREATING" A CLOSURE

secureBooking() EC

passengerCount = 0

```
const secureBooking = function () {  
  let passengerCount = 0;  
  
  return function () {  
    passengerCount++;  
    console.log(`${passengerCount}  
    passengers`);  
  };  
};  
  
const booker = secureBooking();
```

Global scope

secureBooking = <f>
booker = <f>

secureBooking() scope

passengerCount = 0

secureBooking = <f>
booker = <f>

Global EC

secureBooking = <f>
booker = <f>

CALL STACK

Order in which
functions were *called*

Order in which functions
are *written in the code*

SCOPE CHAIN

UNDERSTANDING CLOSURES

secureBooking() EC

passengerCount = 0

- 👉 Variable environment (VE) that popped off stack after secureBooking
- 👉 Because of the closure, VE was **moved to heap** and **NOT garbage collected**
- 👉 This execution context is where **booker** function was created!

booker() EC

<empty>

Global EC

secureBooking = <f>
booker = <f>

CALL STACK

```
const secureBooking = function () {  
  let passengerCount = 0;  
  
  return function () {  
    passengerCount++;  
    console.log(`${passengerCount}  
    passengers`);  
  };  
};  
  
const booker = secureBooking();  
  
booker(); // 1 passengers  
booker(); // 2 passengers
```

This is the function

Global scope

secureBooking = <f>
booker = <f>

secureBooking() scope

passengerCount = 0

secureBooking = <f>
booker = <f>

booker() scope

<empty>

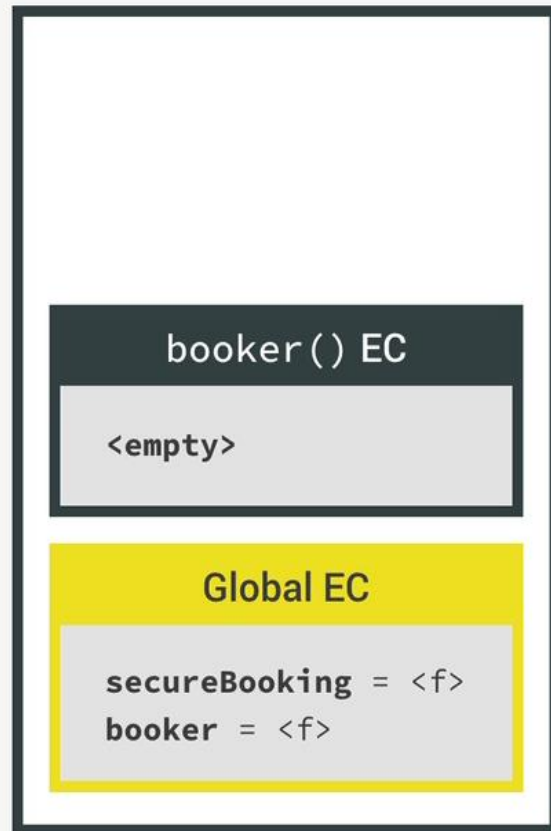
secureBooking = <f>
booker = <f>

How to access
passengerCount?

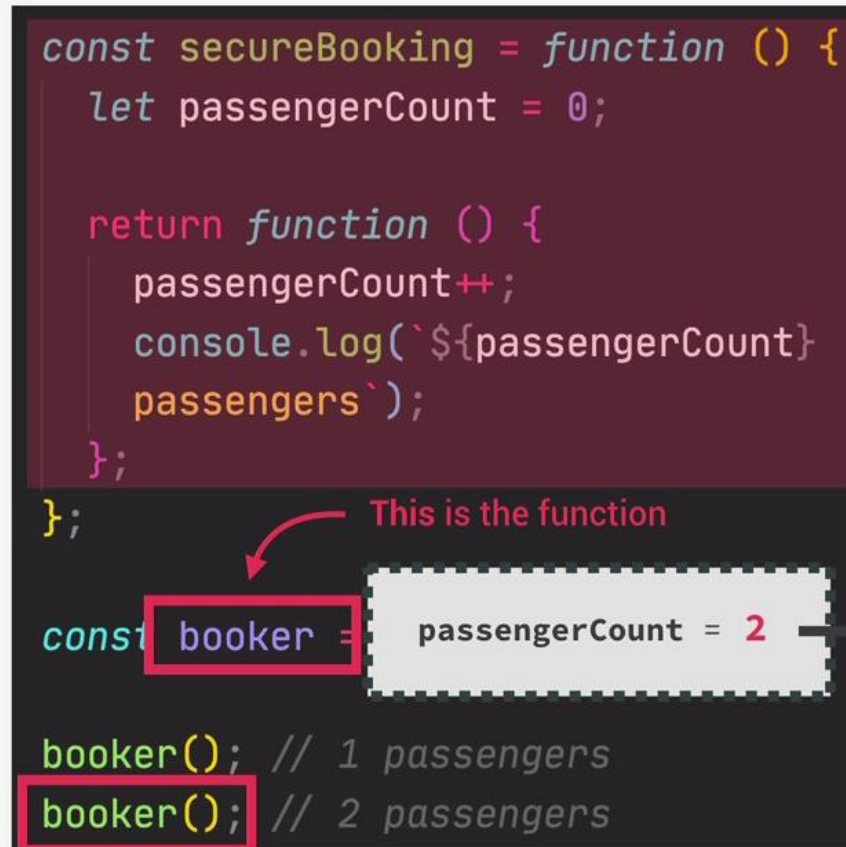
SCOPE CHAIN

UNDERSTANDING CLOSURES

- 👉 A function has access to the variable environment (VE) of the execution context in which it was created
- 👉 **Closure:** VE attached to the function, exactly as it was at the time and place the function was created

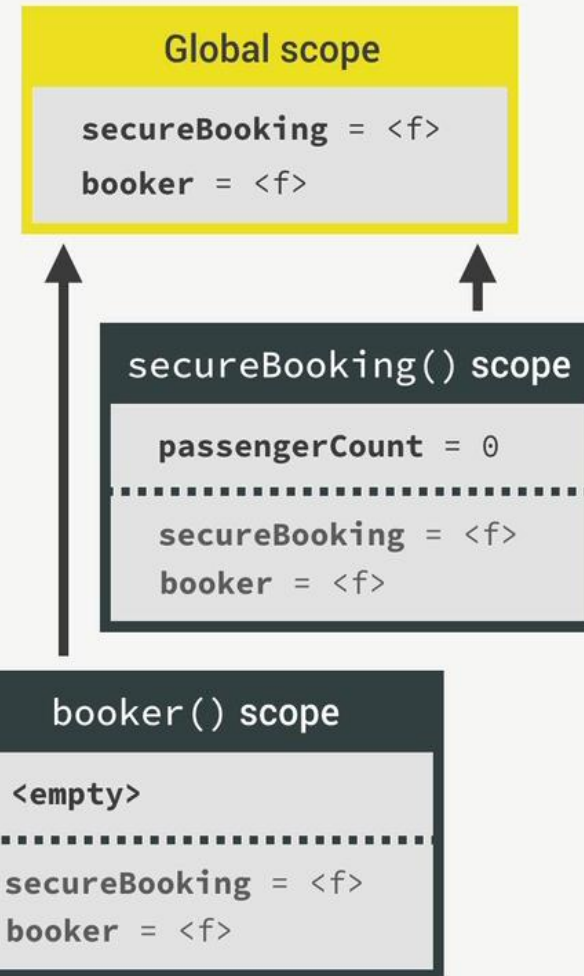


CALL STACK



CLOSURE

How to access
passengerCount?



SCOPE CHAIN

CLOSURES SUMMARY 🥳

- 👉 A closure is the closed-over **variable environment** of the execution context **in which a function was created**, even **after** that execution context is gone;

↓ Less formal

- 👉 A closure gives a function access to all the variables **of its parent function**, even **after** that parent function has returned. The function keeps a **reference** to its outer scope, which **preserves** the scope chain throughout time.

↓ Less formal

- 👉 A closure makes sure that a function doesn't lose connection to **variables that existed at the function's birth place**;

↓ Less formal



- 👉 A closure is like a **backpack** that a function carries around wherever it goes. This backpack has all the **variables that were present in the environment where the function was created**.



- 👉 We do **NOT** have to manually create closures, this is a JavaScript feature that happens automatically. We can't even access closed-over variables explicitly. A closure is **NOT** a tangible JavaScript object.

References

<https://medium.com/@dipikagandash411/deep-dive-into-v8-engine-fb4c2a47e120>